Instituto Tecnológico de Costa Rica

Centro Académico de Alajuela



Diseño de Software

Proyecto Etapa 1

Grupo:

Jeremy Josué Hidalgo Soto - Carnet: 2024134178

Santiago Ballestero – Carnet: 2024150812

Kendall Chacón – Carnet: 2024122026

Profesora: Raquel Rojas

Fecha de entrega: 18 Octubre, 2026

1. Documento de visión

a. Nombre del proyecto

MiniGestor – Sistema para administrar un minisúper

b. Problema

En muchos minisúper pequeños, el control de productos, inventario y ventas se hace de forma manual o con información que está separada en diferentes lugares.

Esto puede causar problemas como no saber cuántos productos quedan, quedarse sin productos importantes, perder el control de las ventas o no saber cuáles productos se venden más.

La idea del sistema es ayudar a tener toda esta información más ordenada y fácil de consultar.

c. Contexto

El sistema está pensado para un minisúper que necesita llevar un mejor control de sus productos, ventas, inventario y proveedores.

En el negocio se realizan ventas todos los días y constantemente entran y salen productos. Por eso, es importante tener una herramienta que permita saber qué productos hay disponibles, cuáles se están acabando y cómo van las ventas.

La aplicación permitirá guardar toda esta información en un solo lugar.

d. Usuarios

El sistema tendrá principalmente dos tipos de usuarios:

Administrador: podrá manejar productos, inventario, proveedores, usuarios y consultar las ventas.

Cajero: podrá buscar productos y registrar las ventas realizadas.

e. Objetivo

Crear una aplicación web que ayude a administrar de forma sencilla los productos, ventas, inventario y proveedores de un minisúper.

Con esto se busca tener la información más organizada y facilitar el trabajo diario del negocio.

f. Alcance

El sistema permitirá:

- Iniciar sesión.
- Registrar productos.
- Modificar productos.
- Consultar productos.
- Organizar productos por categorías.
- Registrar proveedores.
- Consultar el inventario.
- Registrar ventas.
- Actualizar el inventario cuando se realice una venta.
- Consultar ventas anteriores.
- Mostrar productos con poco inventario.
- Importar productos desde un archivo.
- Recomendar productos que deberían comprarse nuevamente.

g. Fuera de alcance

Para no hacer el proyecto demasiado grande, inicialmente no se incluirán funciones como:

- Facturación electrónica.
- Pagos con tarjeta.
- Pagos con SINPE.
- Aplicación móvil.
- Tienda en línea.
- Entregas a domicilio.
- Manejo de salarios.
- Contabilidad completa.
- Conexión directa con bancos.

La idea es mantener el proyecto sencillo y que se pueda terminar correctamente dentro del tiempo que nos dio la profe.

h. Solución propuesta

Para desarrollar el sistema se propone crear una aplicación web llamada MiniGestor. La parte visual será desarrollada con React, mientras que la lógica del sistema se realizará con Python y Django. Para comunicar React con Django se utilizará Django REST Framework mediante una API REST. La información del sistema se almacenará en una base de datos de postgreSQL.

La estructura sería:

React

↓

API REST / JSON

↓

Python + Django

Django REST Framework

↓

poosgreSQL

También se incluirá un proceso ETL para importar información y un agente que ayude a recomendar cuándo sería necesario comprar más productos.

i. Principales funcionalidades

Las principales funciones del sistema serán:

1. **Inicio de sesión:** permitir que los usuarios entren al sistema.
2. **Gestión de productos:** agregar, modificar, consultar y desactivar productos.
3. **Gestión de categorías:** organizar los productos por categorías.
4. **Gestión de proveedores:** registrar y consultar proveedores.
5. **Control de inventario:** revisar cuántos productos hay disponibles.
6. **Registro de ventas:** registrar las ventas realizadas.
7. **Actualización del inventario:** disminuir la cantidad de productos después de una venta.
8. **Historial de ventas:** consultar las ventas realizadas anteriormente.
9. **Alertas de inventario:** avisar cuando un producto tenga pocas unidades.
10. **Importación de productos:** cargar información desde un archivo.
11. **Recomendaciones:** ayudar a decidir cuáles productos deberían comprarse nuevamente.

2. Identificación de actores y roles

a. Actores

Para MiniGestor tendremos estos actores principales:

- **Administrador**
- **Cajero**
- **Sistema externo de archivos**
- **Agente de reposición**

b. Usuarios

Los usuarios son las personas que van a utilizar directamente el sistema.

Cajero

El cajero será el encargado principalmente de registrar las ventas del minisúper.

Podrá:

- Iniciar sesión.
- Buscar productos.
- Consultar precios.
- Registrar ventas.
- Ver el total de una venta.
- Consultar ventas realizadas por él.

No tendrá permiso para cambiar información importante del sistema, como usuarios o proveedores.

c. Administrador

El administrador tendrá más permisos que el cajero.

Será la persona encargada de controlar y mantener la información general del sistema.

Podrá:

- Iniciar sesión.
- Registrar productos.

- Modificar productos.
- Desactivar productos.
- Crear categorías.
- Registrar proveedores.
- Consultar inventario.
- Actualizar inventario.
- Consultar todas las ventas.
- Administrar usuarios.
- Importar productos desde archivos.
- Consultar recomendaciones de compra.

d. Sistemas externos

Para este proyecto no necesitamos conectar demasiados sistemas externos.

Podemos utilizar solamente algo más fácil como:

Archivo CSV

El sistema permitirá cargar archivos CSV con información de productos.

e. Agentes

Nuestro agente será:

Agente de reposición de inventario

Su función será revisar la cantidad de productos disponibles y las ventas realizadas.

El agente ayudará a detectar productos que probablemente deban comprarse nuevamente

f. Matriz de permisos y responsabilidades

Función	**Administrador**	**Cajero**	**Agente**
Iniciar sesión	Sí	Sí	No

Consultar productos	Sí	Sí	Sí
Registrar productos	Sí	No	No
Modificar productos	Sí	No	No
Desactivar productos	Sí	No	No
Administrar categorías	Sí	No	No
Registrar proveedores	Sí	No	No
Consultar inventario	Sí	Sí	Sí
Modificar inventario	Sí	No	No
Registrar ventas	Sí	Sí	No
Consultar ventas	Sí	Sí	Sí
Administrar usuarios	Sí	No	No
Importar archivo CSV	Sí	No	No
Generar recomendaciones	No	No	Sí
Consultar recomendaciones	Sí	No	Sí

Responsabilidades principales

Administrador: se encarga de mantener la información del sistema actualizada y revisar el estado general del minisúper.

Cajero: se encarga principalmente de registrar correctamente las ventas.

Agente de reposición: analiza información del inventario y las ventas para dar recomendaciones sobre cuáles productos podrían necesitar reposición.

Archivo CSV: funciona como una fuente externa de información para importar productos al sistema.

3. Requerimientos funcionales

ID	Descripción	Actor	Prioridad	Criterio de aceptación
RF-01	El sistema debe permitir iniciar sesión con usuario y contraseña.	Administrador / Cajero	Alta	El usuario puede ingresar al sistema si los datos son correctos.
RF-02	El sistema debe permitir registrar nuevos productos.	Administrador	Alta	El producto queda guardado con nombre, precio, categoría y cantidad.
RF-03	El sistema debe permitir modificar la información de un producto.	Administrador	Alta	Los cambios realizados quedan guardados correctamente.
RF-04	El sistema debe permitir desactivar productos.	Administrador	Media	El producto deja de aparecer como disponible para nuevas ventas.
RF-05	El sistema debe permitir registrar y administrar categorías de productos.	Administrador	Media	Se pueden crear, modificar y consultar categorías.
RF-06	El sistema debe permitir registrar proveedores.	Administrador	Media	El proveedor queda guardado con su

				información principal.
RF-07	El sistema debe permitir consultar el inventario disponible.	Administrador / Cajero	Alta	El usuario puede ver la cantidad disponible de cada producto.
RF-08	El sistema debe permitir registrar ventas.	Administrador / Cajero	Alta	La venta queda guardada con los productos, cantidades y total.
RF-09	El sistema debe actualizar el inventario después de realizar una venta.	Sistema	Alta	La cantidad vendida se descuenta automáticamente del inventario.
RF-10	El sistema debe permitir consultar el historial de ventas.	Administrador / Cajero	Media	Se pueden consultar las ventas realizadas anteriormente.
RF-11	El sistema debe mostrar una alerta cuando un producto tenga poco inventario.	Administrador	Alta	El sistema identifica y muestra los productos que llegaron al nivel mínimo definido.
RF-12	El sistema debe permitir importar productos desde un archivo CSV.	Administrador	Media	Los productos válidos del archivo se cargan correctamente en la base de datos.

RF-13	El sistema debe generar recomendaciones sobre productos que necesitan reposición.	Agente	Media	El agente muestra recomendaciones usando el inventario y las ventas registradas.

4. Requerimientos no funcionales

ID	**Tipo**	**Descripción**	**Criterio de aceptación**
RNF-01	Seguridad	El sistema debe proteger el acceso a la información mediante usuario y contraseña.	Un usuario sin credenciales válidas no puede ingresar al sistema.
RNF-02	Usabilidad	La aplicación debe ser sencilla de usar y fácil de entender.	Un usuario puede encontrar las funciones principales sin necesidad de ayuda técnica.
RNF-03	Rendimiento	El sistema debe responder rápidamente a las acciones normales del usuario.	Las consultas comunes, como buscar productos o revisar inventario, deben responder en menos de 3 segundos.
RNF-04	Disponibilidad	El sistema debe estar disponible durante el horario normal de trabajo del minisúper.	Los usuarios pueden utilizar el sistema durante la jornada sin

			interrupciones frecuentes.
RNF-05	Mantenibilidad	El código debe estar organizado por módulos para facilitar cambios futuros.	Las funciones de ventas, productos, inventario y usuarios estarán separadas dentro del backend.
RNF-06	Escalabilidad	El sistema debe permitir aumentar la cantidad de productos, ventas e información registrada sin tener que modificar completamente su estructura.	El sistema puede manejar un aumento en la cantidad de información sin modificar su estructura principal.
RNF-07	Portabilidad	La aplicación debe poder utilizarse desde diferentes computadoras mediante un navegador web.	El sistema funciona correctamente en navegadores modernos como Chrome, Edge o Firefox.
RNF-08	Testabilidad	Las funciones principales del sistema deben poder probarse de forma separada.	Se pueden realizar pruebas sobre funciones como registrar productos, ventas y usuarios sin depender de toda la aplicación.

5. Historias de Usuario

HU-01 – Iniciar sesión

Como usuario,
quiero iniciar sesión con mi usuario y contraseña,
para poder entrar al sistema según mis permisos.

Criterios de aceptación:

- El usuario debe ingresar un usuario y contraseña.
- Si los datos son correctos, puede entrar al sistema.
- Si los datos son incorrectos, se debe mostrar un mensaje de error.

HU-02 – Registrar productos

Como administrador,
quiero registrar nuevos productos,
para mantener actualizado el catálogo del minisúper.

Criterios de aceptación:

- Se debe ingresar nombre, precio, categoría y cantidad.
- El sistema debe validar que los datos estén completos.
- El producto debe quedar guardado correctamente.

HU-03 – Modificar productos

Como administrador,
quiero modificar la información de un producto,
para mantener sus datos actualizados.

Criterios de aceptación:

- El administrador puede seleccionar un producto.
- Puede cambiar datos como precio, nombre o categoría.
- Los cambios deben quedar guardados.

HU-04 – Consultar inventario

Como administrador o cajero,
quiero consultar el inventario,
para saber cuántas unidades hay disponibles de cada producto.

Criterios de aceptación:

- Se debe mostrar la lista de productos.
- Se debe mostrar la cantidad disponible.
- La información debe estar actualizada.

HU-05 – Registrar una venta

Como cajero,
quiero registrar una venta,
para llevar el control de los productos vendidos.

Criterios de aceptación:

- Se pueden seleccionar uno o varios productos.
- Se debe indicar la cantidad vendida.
- El sistema debe calcular el total de la venta.
- La venta debe quedar registrada.

HU-06 – Actualizar inventario automáticamente

Como cajero,
quiero que el inventario se actualice después de una venta,
para no tener que cambiar las cantidades manualmente.

Criterios de aceptación:

- Al registrar una venta, se descuentan las unidades vendidas.
- El sistema no debe permitir vender más unidades de las disponibles.
- El nuevo inventario debe quedar guardado.

HU-07 – Consultar historial de ventas

Como administrador,
quiero consultar las ventas realizadas,
para revisar el movimiento del negocio.

Criterios de aceptación:

- Se debe mostrar una lista de ventas.
- Cada venta debe mostrar fecha, productos y total.
- El administrador puede consultar ventas anteriores.

HU-08 – Registrar proveedores

Como administrador,
quiero registrar proveedores,
para mantener organizada la información de las personas o empresas que venden productos al minisúper.

Criterios de aceptación:

- Se pueden registrar datos básicos del proveedor.
- El proveedor queda guardado.
- El administrador puede consultar los proveedores registrados.

HU-09 – Recibir alertas de inventario

Como administrador,
quiero recibir una alerta cuando un producto tenga pocas unidades,
para saber cuándo debería volver a comprarlo.

Criterios de aceptación:

- Cada producto puede tener una cantidad mínima definida.
- Cuando llegue a esa cantidad, el sistema debe mostrar una alerta.
- La alerta debe indicar cuál producto necesita atención.

HU-10 – Importar productos desde archivo

Como administrador,
quiero importar productos desde un archivo CSV,
para registrar varios productos de una forma más rápida.

Criterios de aceptación:

- El sistema debe permitir seleccionar un archivo CSV.
- Los datos deben ser revisados antes de guardarse.
- Los productos correctos deben cargarse en la base de datos.
- Si existen errores, el sistema debe indicarlos.

HU-11 – Obtener recomendaciones de reposición

Como administrador,
quiero recibir recomendaciones sobre cuáles productos debería comprar nuevamente,
para evitar quedarme sin productos importantes.

Criterios de aceptación:

- El agente debe revisar el inventario.
- El agente debe tomar en cuenta las ventas registradas.
- El sistema debe mostrar una recomendación al administrador.
- La recomendación no debe realizar una compra automáticamente.

HU-12 – Administrar usuarios

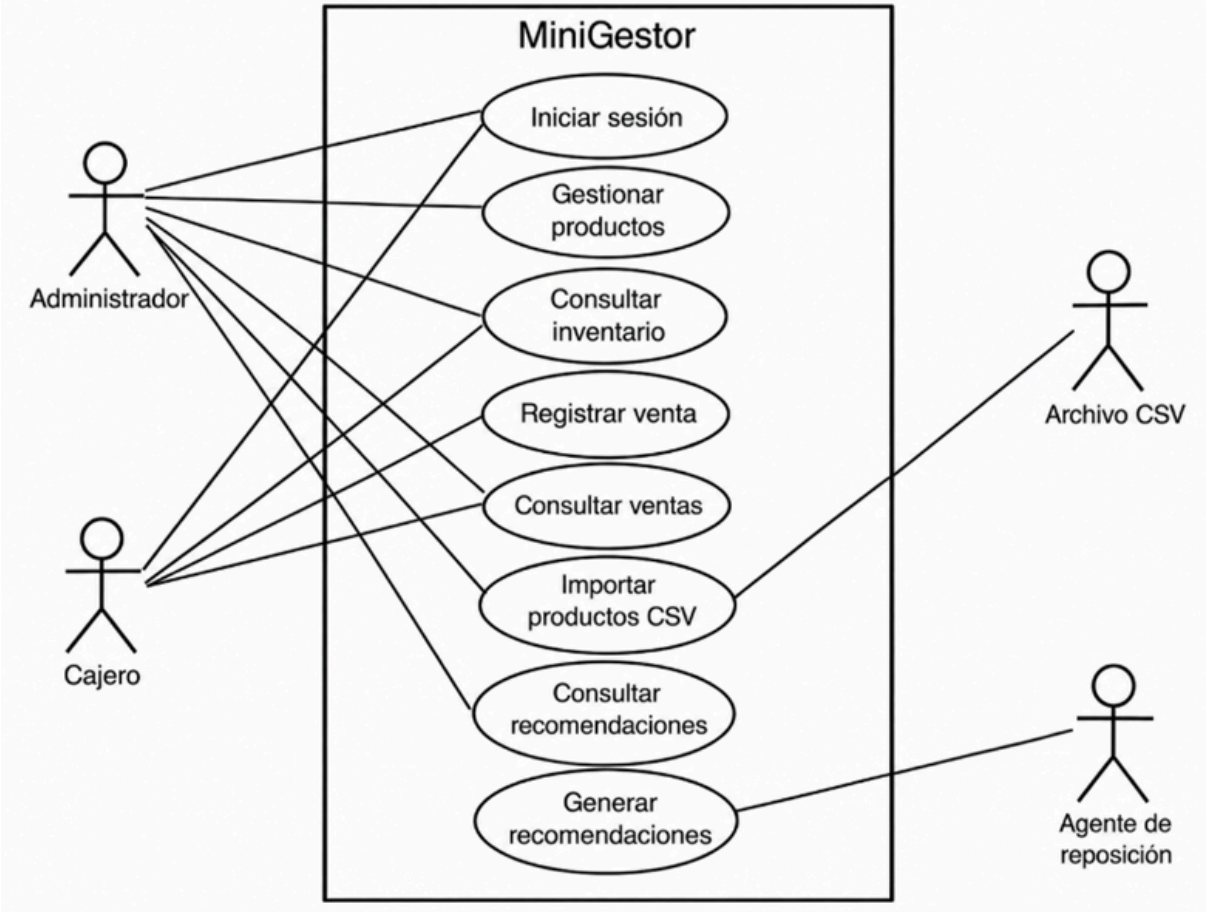
Como administrador,
quiero registrar y administrar usuarios,
para controlar quién puede utilizar el sistema.

Criterios de aceptación:

- El administrador puede crear usuarios.
- Puede asignar un rol como administrador o cajero.
- Los usuarios deben tener permisos según su rol.

6. Casos de Uso

6.1 Diagrama de casos de uso



6.2 Descripción de los casos de uso principales

CU-01 – Iniciar sesión

Actor: Administrador / Cajero

Objetivo: Permitir que el usuario ingrese al sistema.

Flujo principal:

1. El usuario abre la aplicación.
2. El sistema muestra la pantalla de inicio de sesión.
3. El usuario escribe su usuario y contraseña.
4. El sistema revisa los datos.

5. Si son correctos, permite el ingreso.
6. El sistema muestra las opciones según el rol del usuario.

Flujo alternativo:

- Si el usuario o contraseña son incorrectos, el sistema muestra un mensaje de error.
- El usuario puede intentar ingresar nuevamente.

CU-02 – Gestionar productos

Actor: Administrador

Objetivo: Mantener actualizada la información de los productos.

Flujo principal:

1. El administrador entra a la sección de productos.
2. El sistema muestra los productos registrados.
3. El administrador selecciona registrar un nuevo producto.
4. Ingresa nombre, precio, categoría, cantidad y demás información.
5. El sistema revisa los datos.
6. El producto queda guardado.

También se podrá modificar o desactivar un producto existente.

Flujo alternativo:

- Si falta información obligatoria, el sistema muestra un mensaje.
- Si el producto ya existe, el sistema informa al administrador.
- Si el precio o cantidad no son válidos, no se guarda el producto.

CU-03 – Consultar inventario

Actor: Administrador / Cajero

Objetivo: Consultar las cantidades disponibles de los productos.

Flujo principal:

1. El usuario entra a la sección de inventario.
2. El sistema consulta los productos.
3. Se muestra el nombre de cada producto y su cantidad disponible.
4. El usuario puede buscar un producto específico.
5. El sistema muestra la información actualizada.

Flujo alternativo:

- Si no existen productos registrados, se muestra un mensaje indicando que el inventario está vacío.
- Si la búsqueda no encuentra resultados, el sistema informa al usuario.

CU-04 – Registrar venta

Actor: Cajero / Administrador

Objetivo: Registrar una venta realizada en el minisúper.

Flujo principal:

1. El usuario selecciona la opción de nueva venta.
2. Busca o selecciona un producto.
3. Indica la cantidad.
4. El sistema revisa que haya suficientes unidades.
5. El producto se agrega a la venta.
6. Se pueden agregar más productos.
7. El sistema calcula el total.
8. El usuario confirma la venta.
9. El sistema guarda la venta.
10. El inventario se actualiza automáticamente.

Flujo alternativo:

- Si no hay suficientes unidades, el sistema muestra una alerta.

- Si el producto está desactivado, no se puede vender.
- Si se cancela la venta, no se modifica el inventario.

CU-05 – Importar productos desde CSV

Actor: Administrador

Actor externo: Archivo CSV

Objetivo: Permitir registrar varios productos de una manera más rápida.

Flujo principal:

1. El administrador entra a la opción de importar productos.
2. Selecciona un archivo CSV.
3. El sistema lee la información.
4. El sistema revisa los datos.
5. Se corrigen o transforman los datos necesarios.
6. Los productos válidos se guardan en la base de datos.
7. El sistema muestra el resultado de la importación.

Este caso de uso será parte del **ETL** del proyecto.

Flujo alternativo:

- Si el archivo no tiene el formato correcto, se muestra un error.
- Si algún producto contiene información incorrecta, no se carga ese registro.
- El sistema debe indicar cuáles filas tuvieron problemas.

CU-06 – Generar recomendaciones de reposición

Actor: Agente de reposición

Actor relacionado: Administrador

Objetivo: Ayudar a identificar productos que posiblemente necesitan comprarse nuevamente.

Flujo principal:

1. El agente consulta el inventario.
2. Consulta las ventas registradas.
3. Revisa cuáles productos tienen pocas unidades.
4. Analiza cuáles productos tienen mayor movimiento.
5. Genera recomendaciones.
6. Las recomendaciones quedan disponibles para el administrador.

Flujo alternativo:

- Si no existen suficientes datos de ventas, el agente puede basarse únicamente en el nivel mínimo del inventario.
- Si un producto tiene suficiente inventario, no se genera una alerta de reposición.

CU-07 – Consultar historial de ventas

Actor: Administrador / Cajero

Objetivo: Consultar las ventas realizadas anteriormente.

Flujo principal:

1. El usuario entra al historial de ventas.
2. El sistema muestra las ventas registradas.
3. Cada venta muestra fecha, productos, cantidades y total.
4. El usuario puede seleccionar una venta para ver más detalles.

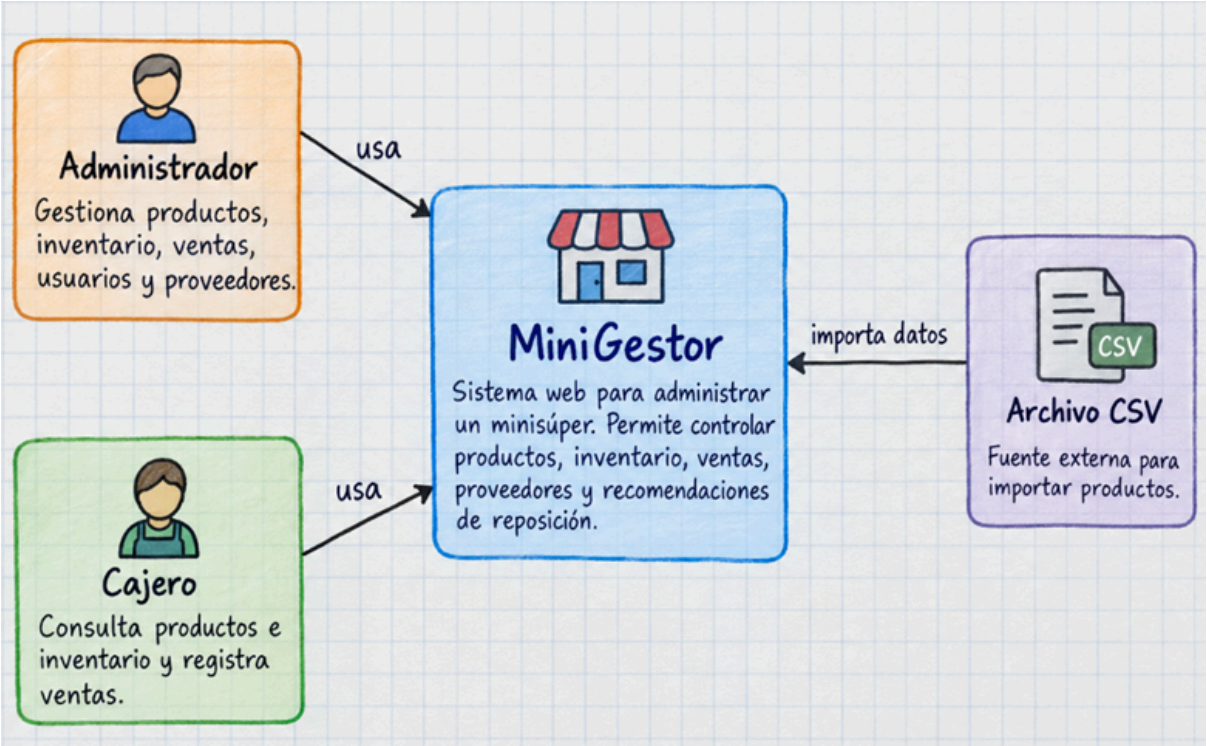
Flujo alternativo:

- Si todavía no existen ventas, el sistema muestra que no hay información disponible.
- El cajero solo podrá consultar las ventas que tenga permitido visualizar según su rol.

7. Arquitectura

7.1 C4 Nivel 1 – Diagrama de Contexto

Este diagrama muestra de forma general quiénes usan el sistema y con qué elementos externos se relaciona. En MiniGestor, el administrador y el cajero usan la aplicación para realizar sus tareas. También aparece el archivo CSV, que se utiliza para importar información de productos. La idea es mostrar el sistema desde afuera, sin entrar todavía en detalles técnicos.

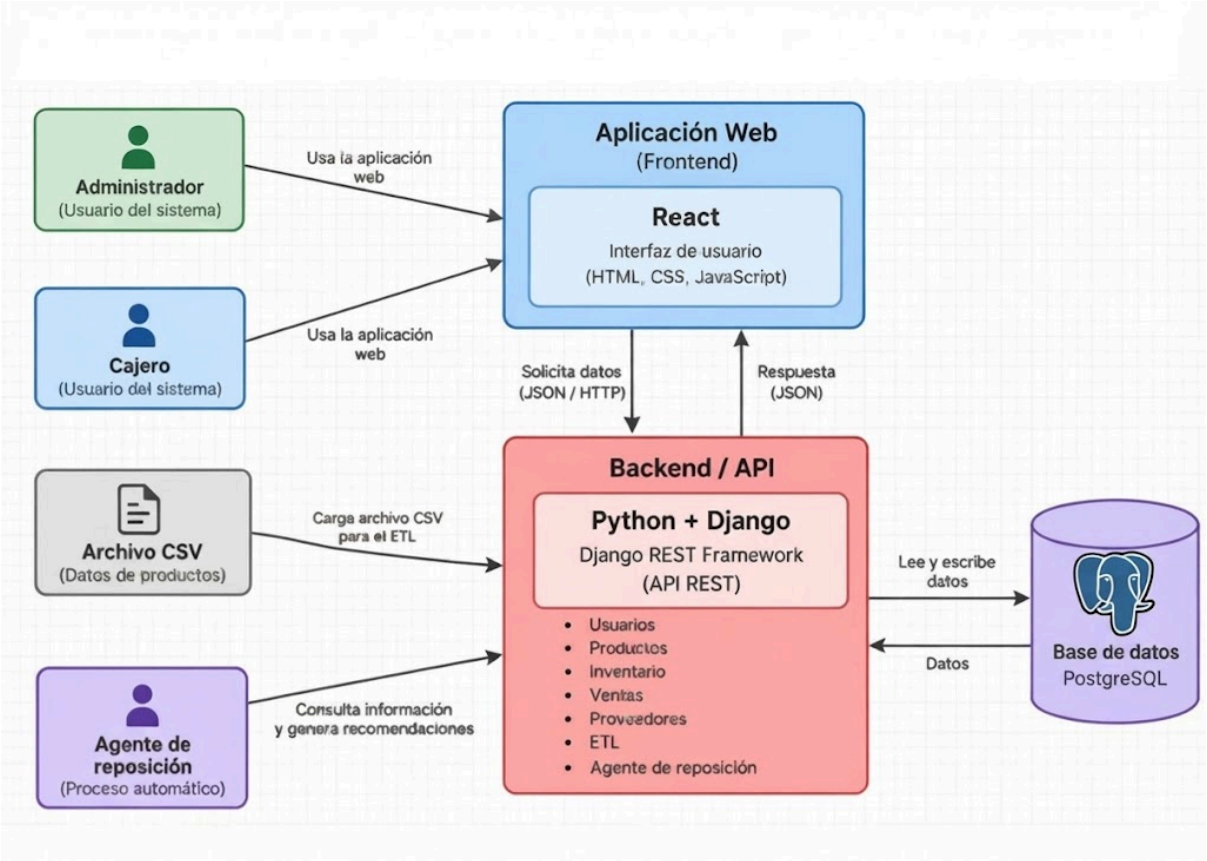


7.2 C4 Nivel 2 – Diagrama de Contenedores

Este diagrama muestra las partes principales que forman MiniGestor.

La parte visual está hecha con React, que es lo que ve y usa la persona. Luego está el backend, hecho con Python y Django, que se encarga de procesar la información y aplicar las reglas del sistema. Para comunicar React con Django se utiliza una API REST.

Finalmente, la información se guarda en PostgreSQL. También se muestra el archivo CSV, que se usa para cargar productos mediante el proceso ETL.



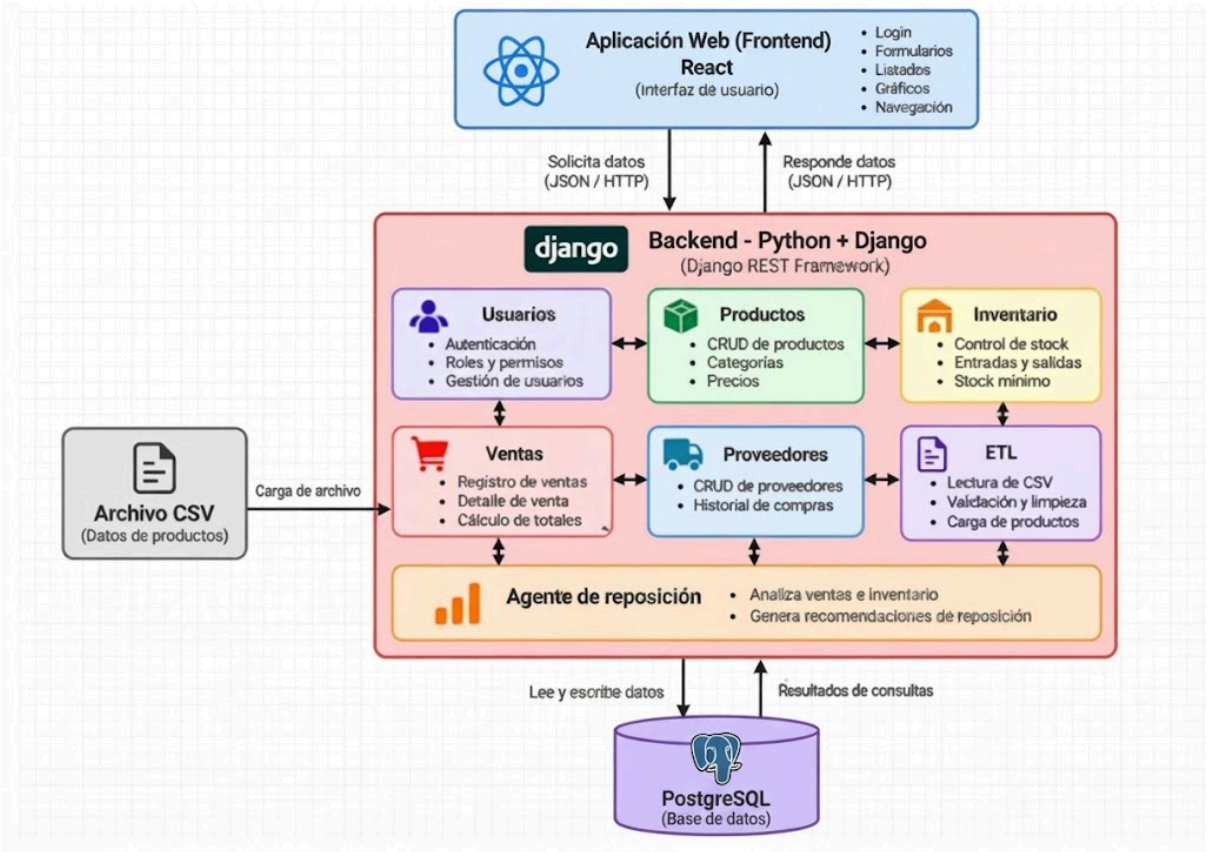
7.3 C4 Nivel 3 – Diagrama de Componentes

Este diagrama muestra con más detalle cómo está organizado el backend de Django por dentro.

El sistema se divide en diferentes partes, como usuarios, productos, inventario, ventas, proveedores, ETL y el agente de reposición.

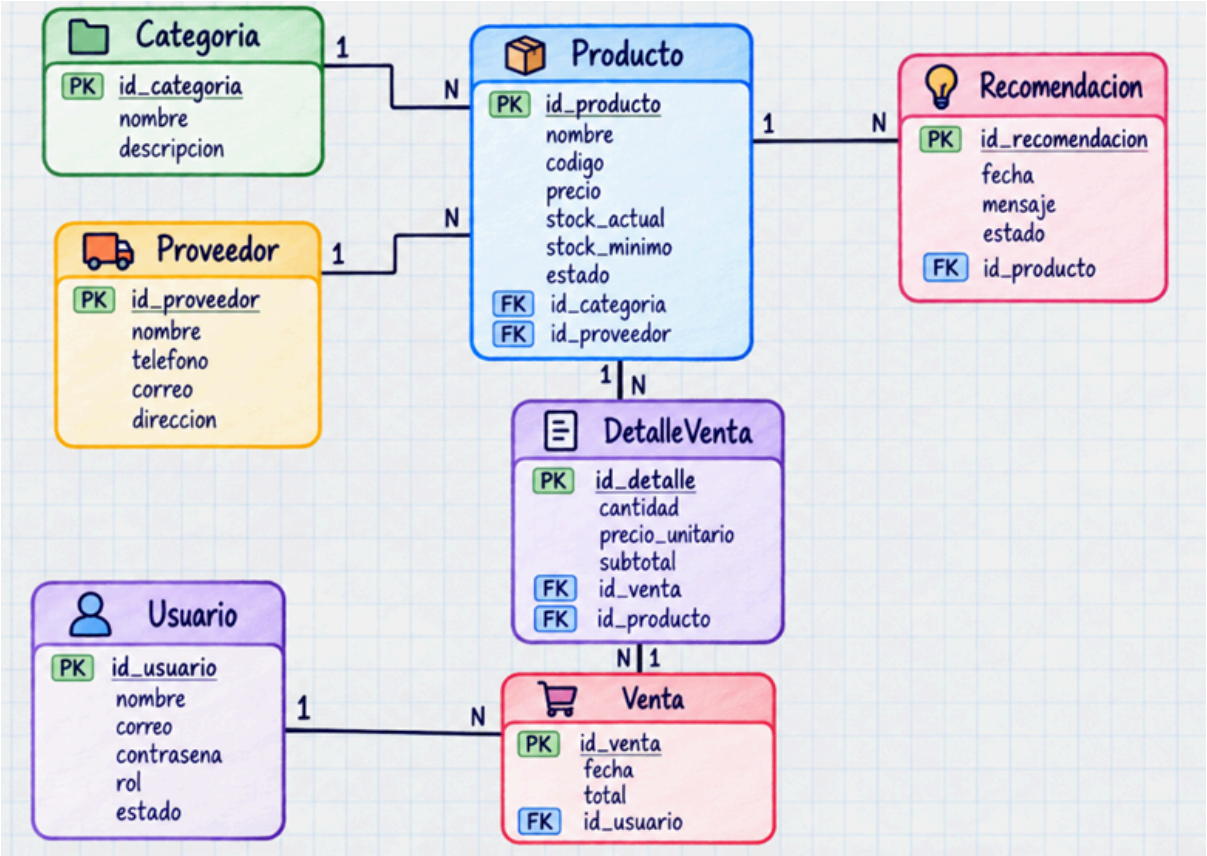
Cada parte tiene una función diferente. Por ejemplo, ventas registra las compras realizadas, inventario controla las existencias y el agente analiza la información para generar recomendaciones.

La idea es mostrar cómo se organiza internamente el sistema para que cada parte tenga una responsabilidad clara.


Aplicación Web (Frontend)
React
(Interfaz de usuario)
Login
Formularios
Listados
Gráficos
Navegación
Solicita datos
(JSON / HTTP)
Responde datos
(JSON / HTTP)
django
Backend - Python + Django
(Django REST Framework)
Usuarios
Autenticación
Roles y permisos
Gestión de usuarios
Productos
CRUD de productos
Categorías
Precios
Inventario
Control de stock
Entradas y salidas
Stock mínimo
Archivo CSV
(Datos de productos)
Carga de archivo
Ventas
Registro de ventas
Detalle de venta
Cálculo de totales
Proveedores
CRUD de proveedores
Historial de compras
ETL
Lectura de CSV
Validación y limpieza
Carga de productos
Agente de reposición
Analiza ventas e inventario
Genera recomendaciones de reposición
Lee y escribe datos
Resultados de consultas
PostgreSQL
(Base de datos)

8. Modelo de datos

A) Diagrama ER:



i. Entidades

Las entidades principales del sistema son:

- Usuario
- Categoría
- Proveedor
- Producto
- Venta
- DetalleVenta
- Recomendación

ii. Atributos

Usuario

- id_usuario

- nombre
- correo
- contraseña
- rol
- estado

Categoría

- id_categoria
- nombre
- descripción

Proveedor

- id_proveedor
- nombre
- teléfono
- correo
- dirección

Producto

- id_producto
- nombre
- código
- precio
- stock_actual
- stock_minimo
- estado
- id_categoria
- id_proveedor

Venta

- id_venta
- fecha
- total
- id_usuario

DetalleVenta

- id_detalle
- cantidad
- precio_unitario
- subtotal

- id_venta
- id_producto

Recomendación

- id_recomendacion
- fecha
- mensaje
- estado
- id_producto

iii. PK

- Usuario → **id_usuario**
- Categoría → **id_categoria**
- Proveedor → **id_proveedor**
- Producto → **id_producto**
- Venta → **id_venta**
- DetalleVenta → **id_detalle**
- Recomendación → **id_recomendacion**

iv. FK

- Producto → **id_categoria**
- Producto → **id_proveedor**
- Venta → **id_usuario**
- DetalleVenta → **id_venta**
- DetalleVenta → **id_producto**
- Recomendación → **id_producto**

v. Relaciones

Las relaciones entre las entidades son:

- Una categoría puede tener varios productos.
- Un proveedor puede estar relacionado con varios productos.
- Un usuario puede registrar varias ventas.
- Una venta puede tener varios detalles de venta.
- Un producto puede aparecer en varios detalles de venta.
- Un producto puede tener varias recomendaciones.

vi. Cardinalidades

Las cardinalidades quedarían así:

Relación	Cardinalidad
Categoría → Producto	1 : N
Proveedor → Producto	1 : N
Usuario → Venta	1 : N
Venta → DetalleVenta	1 : N
Producto → DetalleVenta	1 : N
Producto → Recomendación	1 : N

9. Arquitectura Propuesta

Para MiniGestor se utilizará una arquitectura cliente-servidor, junto con un monolito modular para organizar la parte del backend.

La parte visual será desarrollada con React, donde el administrador y el cajero podrán utilizar las diferentes funciones del sistema.

La parte lógica será desarrollada con Python y Django. El backend estará dividido en módulos como usuarios, productos, inventario, ventas, proveedores, ETL y agente de reposición.

Para comunicar React con Django se utilizará Django REST Framework por medio de una API REST.

La información del sistema se guardará en una base de datos PostgreSQL.

Se escogió esta arquitectura porque permite mantener el sistema organizado y separar las diferentes funciones sin hacerlo demasiado complicado.

La estructura general será:

Usuario

↓

React

Parte visual del sistema

↓

API REST

Django REST Framework

↓

Python + Django

Lógica del sistema

↓

PostgreSQL

Base de datos

También se organizará el backend por módulos, por ejemplo:

Django

│

├── Usuarios

├── Productos

├── Inventario

├── Ventas

├── Proveedores

├── ETL

└── Agente de reposición

9.1 ¿Por qué escogimos esta arquitectura?

Se escogió porque es sencilla de entender y permite separar las funciones del sistema.

React se encarga de la parte visual, Django se encarga de la lógica y PostgreSQL guarda la información.

Además, al dividir el backend en diferentes módulos, es más fácil trabajar cada parte por separado y hacer cambios sin afectar todo el sistema.

Esta arquitectura también es adecuada para el tiempo disponible y no agrega una complejidad innecesaria al proyecto, algo que creemos que es muy importante que se tome en cuenta

10. Componentes y Servicios

10.1 Componentes principales

1. Usuarios
Se encarga del inicio de sesión, los roles y los permisos.

2. Productos
Permite registrar, modificar, consultar y desactivar productos.

3. Inventario
Controla la cantidad disponible de cada producto y el stock mínimo.

4. Ventas
Permite registrar ventas, calcular el total y guardar el detalle de los productos vendidos.

5. Proveedores
Permite registrar y consultar la información de los proveedores.

6. ETL
Se encarga de leer archivos CSV, revisar los datos y cargar los productos correctos al sistema.

7. Agente de reposición
Analiza las ventas y el inventario para recomendar cuáles productos deberían comprarse nuevamente.

10.2 Responsabilidades

Cada componente tendrá una función específica. Por ejemplo:

- Ventas se encarga solamente de las ventas.
- Inventario se encarga de las existencias.
- Productos maneja la información de los productos.
- Usuarios controla accesos y permisos.
- ETL se encarga de importar datos.
- El agente genera recomendaciones.

Esto ayuda a mantener el sistema más ordenado.

10.3 Interfaces

La principal forma de comunicación será mediante una API REST, react enviará solicitudes al backend y Django responderá con la información necesaria.

React

↓

API REST

↓

Django

10.4 Servicios

Algunos servicios del sistema serán:

- Servicio de autenticación.
- Servicio de productos.
- Servicio de inventario.
- Servicio de ventas.

- Servicio de proveedores.
- Servicio de importación de CSV.
- Servicio de recomendaciones.

10.5 Dependencias

Algunas partes del sistema necesitan información de otras.

Por ejemplo: Cuando se realiza una venta, el módulo de ventas necesita actualizar el inventario.

Otro ejemplo: El agente necesita revisar tanto las ventas como el inventario, para dar una recomendación

Otro ejemplo: El ETL necesita enviar los productos correctos para que sean guardados en la base PostgreSQL

Principios que se aplican

Cohesión: cada módulo se encarga de tareas relacionadas entre sí.

Bajo acoplamiento: se busca que los módulos no dependan demasiado unos de otros.

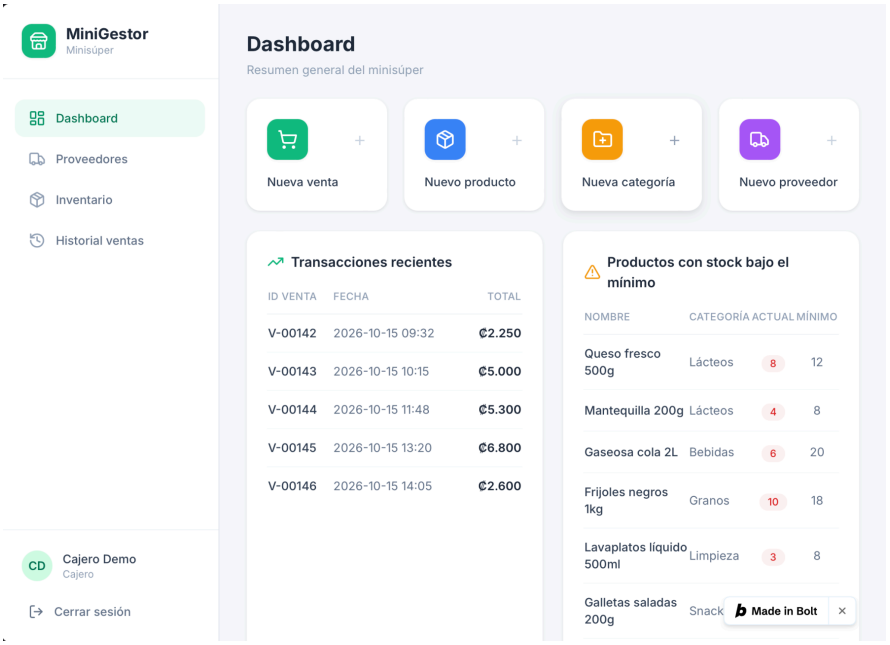
Separación de responsabilidades: cada parte tiene una función definida.

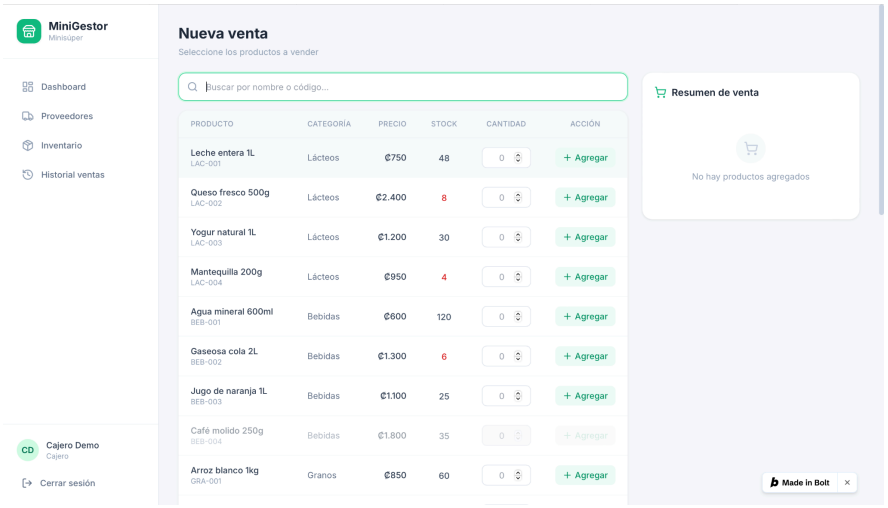
Reutilización: algunas funciones, como validaciones o consultas, pueden utilizarse en varias partes.

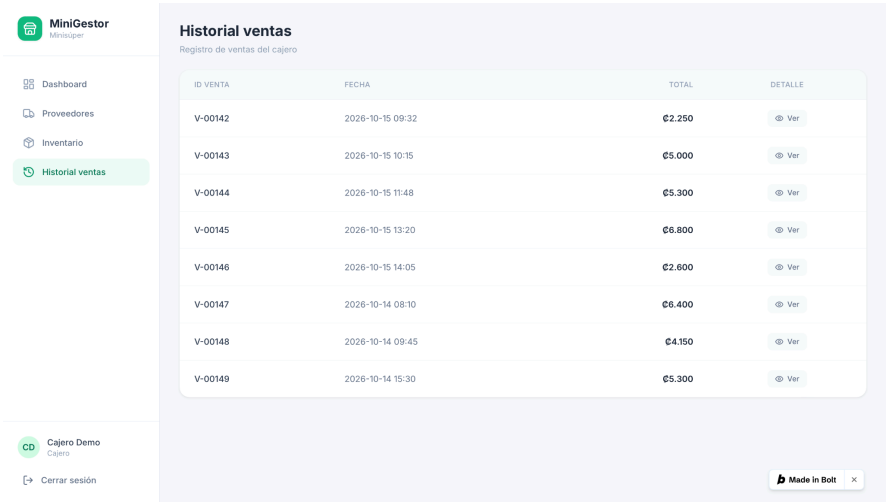
Abstracción: cada módulo puede utilizar otros servicios sin necesitar conocer todos sus detalles internos.

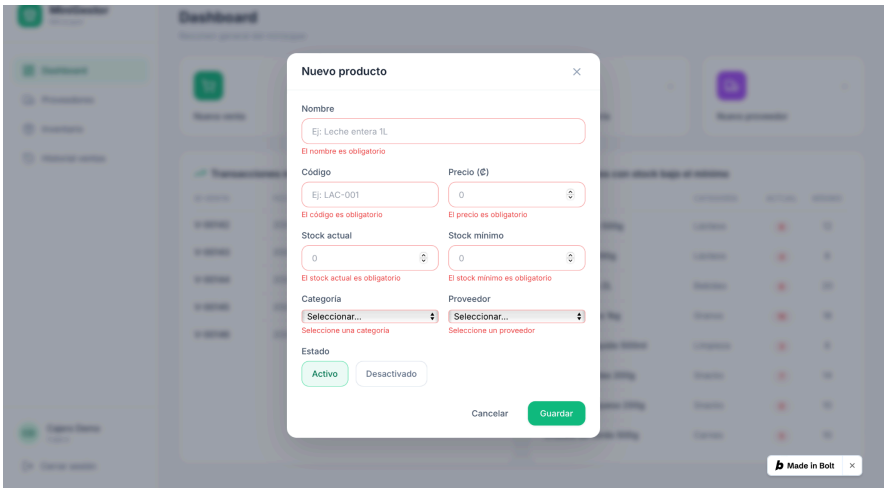
11. Diseño UI/UX

11.1 Capturas del diseño del sistema:









11.2 Consistencia

- El sidebar, el logo y el usuario aparecen igual en todas las pantallas, y la opción activa se resalta en verde.
- Todas las pantallas usan el mismo patrón: título con subtítulo, botón verde de acción arriba a la derecha y contenido en cards blancas con bordes redondeados.
- Las tablas comparten estilo (encabezados en mayúscula gris, filas separadas). Los formatos de moneda (₡2.250) y fecha son iguales en el dashboard, el historial y el detalle.
- Las etiquetas rojas de “Stock bajo” se repiten en inventario, categoría y dashboard.

11.3 Usabilidad

- Visibilidad del estado: menú activo, breadcrumb “Inventario > Lácteos” y alertas de stock visibles.
- Control del usuario: botones “Volver” y “Cancelar”.
- Prevención de errores: el producto desactivado (Café molido) aparece deshabilitado en la venta, y hay un paso de confirmación antes de guardarla.
- Reconocimiento sobre memoria: buscador por nombre o código, y un resumen de venta siempre visible.
- Todo está en español y usa colones, que es lo que el cajero espera ver.

11.4 Responsive Design

MiniGestor contará con un diseño adaptable a pantallas de laptop y desktop, de modo que la interfaz aproveche el ancho disponible del navegador en estos dispositivos (menú lateral fijo, cards y tablas que se ajustan al contenedor). Por decisión de alcance, y para mantener el proyecto sencillo y terminable en el tiempo disponible, no se contempla un diseño adaptable para tablet ni celular. Esta decisión es coherente con el alcance del proyecto, que excluye la aplicación móvil, y con el requerimiento RNF07, que establece que el sistema se utilizará desde diferentes computadoras mediante un navegador web moderno (Chrome, Edge o Firefox).

11.5 Validación

La aplicación final validará la información ingresada por el usuario en todos sus formularios y flujos: credenciales de inicio de sesión contra la base de datos (HU-01, RNF01), campos obligatorios, formato de precio, stock y correo, productos duplicados (CU-02), unidades suficientes en una venta (CU-04) y errores por fila en la importación de CSV (CU-05).

Por cuestiones de simplicidad, el prototipo no valida usuarios: el inicio de sesión acepta cualquier usuario y contraseña, ya que no existe una base de datos de usuarios en esta etapa. Sin embargo, el prototipo sí demuestra el comportamiento de validación ante acciones no permitidas, mostrando mensajes de error claros al usuario.

11.6 Accesibilidad

- No depende solo del color: el stock bajo se indica con una etiqueta de texto (“Stock bajo”) además del color rojo, y el estado de los productos con texto (“Activo”).
- Etiquetas visibles en los formularios: cada campo tiene su etiqueta sobre el input y no solo un texto de ayuda dentro del campo.
- Foco visible: el campo activo se resalta con un borde verde, lo que facilita la navegación con teclado.
- Áreas de clic amplias: botones y filas de tabla tienen un tamaño cómodo para el uso con mouse o trackpad.
- Jerarquía clara: títulos, subtítulos y encabezados de tabla distinguen las secciones de cada pantalla, y el menú resalta la sección activa.
- Iconos acompañados de texto: las opciones del menú y las acciones principales combinan ícono y texto.

Link al prototipo (pegar en buscador):
https://minigestor-desktop-u-oeg6.bolt.host

12. Skills de UI/UX

12.1 Skill 1

- Skill: Generación de prompts para prototipos
- *Propósito:* convertir requisitos y decisiones de diseño en un prompt que una herramienta de IA pueda construir.
- *Entrada:* documento de información del proyecto (requisitos, historias de usuario, modelo de datos) y decisiones de pantallas.
- *Salida:* prompt estructurado por pantallas, modales, datos de ejemplo y reglas.
- *Integración:* IA generativa para diseño de prototipos.
- *Evidencia:* el prompt usado y las capturas del prototipo resultante.

12.2 Skill 2

- Skill: Evaluación heurística
- *Propósito:* detectar problemas de usabilidad antes de programar.
- *Entrada:* capturas de las pantallas.
- *Salida:* hallazgos por heurística de Nielsen con recomendaciones.
- *Integración:* revisión del prototipo.
- *Evidencia:* hallazgos reales, como problemas que se hayan tenido a la hora de diseñar.

12.3 Skill 3

- Skill: Diseño de formularios y validaciones
- *Propósito:* definir qué pide cada formulario y qué errores muestra.
- *Entrada:* una entidad del modelo de datos, por ejemplo Producto.
- *Salida:* campos, tipos, reglas de validación y mensajes de error en español.
- *Integración:* modales de producto, proveedor, categoría y edición.
- *Evidencia:* los formularios.

13. Agente de software

Nombre: Agente de Análisis y Reposición de Inventario (AARI).

Propósito: Analizar los niveles de inventario, la velocidad de rotación de los productos y la información de los proveedores para generar recomendaciones automatizadas y precisas sobre cuándo y cuánto producto reponer, optimizando el flujo de caja y evitando quiebres de stock en el Punto de Venta (POS).

Rol: Analista de Inventario y Asistente de Compras. Actúa como un consultor interno que revisa los datos fríos y entrega sugerencias procesables al usuario administrador.

Entradas:

- Listado de productos en estado crítico (donde `stock_actual` es menor o igual al `stock_minimo`).
- Historial transaccional de ventas recientes cruzado por producto (tablas `Venta` y `DetalleVenta`).
- Información de contacto y catálogo del proveedor asociado a cada producto (tabla `Proveedor`).

Herramientas: El agente utiliza un enfoque de *Tool-Calling* con acceso a las siguientes herramientas (funciones que ejecutan SQL):

1. **`obtener_alertas_stock`**: Retorna los productos que han alcanzado su umbral mínimo.
2. **`calcular_velocidad_ventas`**: Recibe un `id_producto` y calcula el promedio de unidades vendidas por día en los últimos 7, 15 y 30 días.
3. **`obtener_datos_proveedor`**: Recibe un `id_proveedor` y retorna el nombre, teléfono y correo para incluirlo en la recomendación.

Acciones:

1. **Monitoreo:** Ejecuta la herramienta de alertas de stock para identificar productos prioritarios.

2. **Investigación:** Para cada producto prioritario, utiliza la herramienta de velocidad de ventas para proyectar en cuántos días se agotará el inventario restante.
3. **Síntesis:** Cruza la necesidad de reposición con los datos del proveedor correspondiente.
4. **Notificación:** Redacta un mensaje justificado, en lenguaje natural, indicando la cantidad exacta que se sugiere comprar y a quién.

Salidas: Un registro insertado en la tabla `Recomendacion` de la base de datos, que contiene:

- `id_producto`: Referencia al producto analizado.
- `fecha`: Timestamp del análisis.
- `mensaje`: Texto estructurado con la recomendación (Ej: *"El producto X se agotará en 3 días al ritmo actual de ventas. Se sugiere pedir 50 unidades al proveedor Y al teléfono Z"*).
- `estado`: 'Pendiente' (hasta que el usuario interactúe con ella).

Restricciones:

- **Seguridad y Permisos:** El agente tiene acceso estricto de **solo lectura** a las tablas operativas (`Producto`, `Venta`, `DetalleVenta`, `Proveedor`, `Categoria`). Su único permiso de **escritura** está limitado a realizar `INSERT` y `UPDATE` en la tabla `Recomendacion`.
- **Alcance de acción:** El agente es estrictamente consultivo; no puede realizar compras automáticas, enviar correos a proveedores por sí mismo, ni alterar el `stock_actual` de los productos.
- **Límites de contexto:** Se limitará el análisis de histórico de ventas a un máximo de 3 meses para no saturar el contexto del LLM y mantener las recomendaciones basadas en tendencias recientes.

Integración: El agente se integrará en la capa de backend (servidor) del sistema POS. Estará conectado a través del enrutador FreeLLMAPI para procesar el lenguaje natural. Se activará de dos maneras:

1. **Ejecución en segundo plano (Cron Job):** Una vez al día al cierre de caja para dejar las recomendaciones listas para la mañana siguiente.
2. **Ejecución bajo demanda:** Mediante un botón de "Generar Análisis de Inventario" en la interfaz del `Usuario` del POS. Las respuestas se mostrarán en la interfaz gráfica del usuario leyendo la tabla `Recomendacion`.

Uso de inteligencia artificial:

Herramienta utilizada: Se utilizó ChatGPT y su herramienta para generar imágenes.

Propósito: Se utilizó la IA para ayudar a crear los diagramas del proyecto de una forma visual y ordenada, ya que nos toma mucho tiempo que mejor usamos para otras cosas.

Prompt o especificación: Se le indicó a la IA qué tipo de diagrama se necesitaba, qué elementos debía incluir y cómo queríamos que se viera.

Resultado obtenido: Se generaron los diagramas de casos de uso, C4 Nivel 1, C4 Nivel 2, C4 Nivel 3 y el diagrama entidad-relación.

Modificaciones realizadas: Se hicieron cambios en los títulos, colores, distribución de los elementos y estilo de los diagramas para que se adaptaran mejor al proyecto.

Validación realizada: Los integrantes del grupo revisaron que la información de los diagramas fuera correcta y que coincidiera con los requerimientos y el funcionamiento del MiniGestor.

Herramienta utilizada: Bolt

Propósito: Se utilizó para realizar el prototipo visual y los flujos entre pantallas de cómo se vería la aplicación final MiniGestor.

Prompt o especificación:

Crea un prototipo web interactivo de escritorio (layout desktop-first, optimizado para 1440px, NO es necesario que sea responsive ni mobile) para "MiniGestor", un sistema para administrar un minisúper. Usa React + Vite + Tailwind CSS. Es solo un prototipo visual: NO uses backend ni base de datos; usa datos de ejemplo en memoria (estado de React) para que todo sea interactivo. Toda la interfaz va en español.

DISEÑO VISUAL

- Tema CLARO: fondo gris muy suave (#F5F7FA), cards blancas con bordes redondeados y sombra sutil, color de acento verde esmeralda. Tipografía limpia (Inter o Poppins). Estilo de dashboard moderno y limpio.

- Sidebar izquierdo fijo en todas las pantallas (excepto login) con logo "MiniGestor" y 4 opciones: Dashboard, Proveedores, Inventario, Historial ventas. La opción activa se resalta. Abajo del sidebar: nombre del usuario y botón "Cerrar sesión".

PANTALLAS

1) Login

- Pantalla centrada con logo, campos usuario y contraseña, botón "Iniciar sesión". Si los datos están vacíos, muestra mensaje de error. Al entrar, va al Dashboard. (No incluir sign up.)

2) Dashboard

- Encabezado "Dashboard".

- Fila de 4 botones/cards de acceso rápido con ícono y "+": "Nueva venta", "Nuevo producto", "Nueva categoría", "Nuevo proveedor".

  - "Nueva venta" navega a la pantalla de Nueva venta (ver punto 6).

  - "Nuevo producto" abre el modal de producto (ver Modales).

  - "Nueva categoría" abre un modal que pide solo el nombre de la categoría.

  - "Nuevo proveedor" abre el modal de proveedor.

- Sección "Transacciones recientes": tabla con ID de venta, fecha y total (5 filas de ejemplo).

- Sección "Productos con stock bajo el mínimo": lista/tabla con nombre, categoría, stock actual y stock mínimo. Solo aparecen productos cuyo stock actual sea menor al mínimo; si un producto sube por encima del mínimo, desaparece de esta lista automáticamente (calculado a partir del estado).

3) Inventario

- Botón superior "Nuevo producto" que abre un modal con dos opciones (tabs o botones): "Producto" o "Categoría".

  - Si elige Categoría: solo pide el nombre.

  - Si elige Producto: pide el formulario completo de producto.

- Cuadrícula de cards, una por categoría (nombre de la categoría y cantidad de productos). Al hacer clic en una card, NAVEGA a una pantalla nueva (no modal) llamada "Inventario > [Categoría]".

4) Detalle de categoría (pantalla nueva)

- Botón "Volver" a Inventario y título con el nombre de la categoría.

- Tabla de productos de esa categoría con: código, nombre, precio, stock actual, stock mínimo, proveedor y estado (Activo/Desactivado). Si stock actual < stock mínimo, muestra una etiqueta roja "Stock bajo".

- Botón "Editar producto" arriba. Abre un modal en dos pasos: primero un select para elegir el producto de esa categoría; luego un formulario con los campos modificables (nombre, código, precio, stock actual, stock mínimo, categoría, proveedor, estado). Cualquier campo que se deje en blanco NO se modifica. Al guardar, se actualiza la tabla.

5) Proveedores

- Botón superior "Nuevo proveedor" que abre modal con: nombre, teléfono, correo y dirección.

- Tabla con nombre, teléfono, correo y dirección de cada proveedor (6 proveedores de ejemplo). Al guardar un proveedor nuevo, aparece en la tabla.

6) Nueva venta (flujo de 2 pantallas)

- Pantalla 6A "Seleccionar productos": lista de productos parecida a la de inventario, con búsqueda por nombre/código. Cada fila tiene: producto, precio, stock disponible, un campo de cantidad y un botón "Agregar". Los productos desactivados o sin stock aparecen deshabilitados. No se puede agregar más cantidad que el stock disponible (mostrar alerta). Panel lateral o inferior con el resumen: productos agregados, cantidad, subtotal y total, con opción de quitar ítems. Botón "Continuar a registrar venta" (deshabilitado si no hay ítems).

- Pantalla 6B "Registrar venta": muestra el detalle en columnas (producto, cantidad, precio unitario, subtotal), el total, y botones "Confirmar venta" y "Cancelar". Al confirmar: guarda la venta en el estado con ID, fecha y total, DESCUENTA el stock de cada producto, muestra mensaje de éxito y vuelve al Dashboard. Al cancelar, no se modifica el inventario.

7) Historial ventas

- Tabla de las ventas del cajero de la sesión con: ID de venta, fecha y total (8 ejemplos más las que se registren). Al hacer clic en una fila, se abre un modal de solo lectura con el detalle de la venta: productos, cantidades, precio unitario y subtotal.

MODALES (todos con título, botón cerrar, "Cancelar" y "Guardar", validación de campos obligatorios)

- Producto: nombre, código, precio, stock actual, stock mínimo, categoría (select con las categorías existentes), proveedor (select con los proveedores existentes), estado (Activo/Desactivado).

- Categoría: nombre.

- Proveedor: nombre, teléfono, correo, dirección.

DATOS DE EJEMPLO

- Categorías: Lácteos, Bebidas, Granos, Limpieza, Snacks, Carnes.

- 3 a 5 productos por categoría con precios en colones (₡), y al menos 3 productos con stock por debajo del mínimo.

- Un usuario cajero llamado "Cajero Demo".

REGLAS

- Todo el estado (productos, categorías, proveedores, ventas) vive en un contexto de React compartido para que los cambios se reflejen en todas las pantallas.

- Organiza el código en componentes separados (Sidebar, Modal, ProductForm, SupplierForm, tablas, etc.) y una carpeta por pantalla.

- Navegación con React Router.

Resultado obtenido: Se generó el prototipo visual e interactivo de la aplicación MiniGestor.

Modificaciones realizadas: Se hicieron cambios en ciertas pantallas con detalles mínimos como agregar algún componente, agregar algún flujo o eliminar algo que no concordaba con lo especificado.

Validación realizada: Los integrantes del grupo revisaron que el prototipo incluyera todo lo que se estaba pidiendo, que no omitiera puntos importantes y que el diseño de pantallas fuera acorde a lo que se planteó en este documento además de a lo que se requiere para la versión final de la app.

Herramienta utilizada: Google Gemini 3.1 PRO

Propósito: Se utilizó para consultar sobre el repositorio FreeLLMAPI con el fin de ver si vale la pena utilizarlo para la integración del agente de IA

Prompt o especificación:

1.puedo hacer un agente de ia en mi proyecto con este repo? https://github.com/tashfeenahmed/freellmapi

2.es de un proyecto de un minigestor Point of sale, quiero tener un agente de ia que mediante consultas a la base de datos pueda hacer recomendaciones de cuales productos conviene reponer mas rapido en mayor cantidad, cuales no, etc…

3.este es el DER , adaptemos la idea a él

Resultado obtenido: Se generó una propuesta de especificaciones técnicas para el agente de IA del proyecto

Modificaciones realizadas: Se hicieron cambios en las restricciones

Validación realizada: Se hizo una consulta en el grupo y todos los participantes accedieron a aceptar la idea